DDoS Detection Features

What is the Problem with Previous Features proposed in different research papers ?

Using the previously defined features, we identified a major issue known as the dilution of the attack within normal traffic.

In practice, a port that we aim to protect receives traffic simultaneously from many sources. In our case, we are trying to secure a medium-sized service that typically receives between 100,000 and 500,000 packets per second over a given day.

If we use, for example, a feature based on the average throughput computed over all observed traffic within a time window (including both normal traffic and potential attack traffic), the information related to the actual attack rate can be significantly weakened.

Indeed, the large volume of legitimate traffic can mask or dilute the contribution of malicious traffic, making the detection of the attack more difficult.

What is the solution?

To address this problem, it is necessary to design features that are robust to this dilution effect.

Therefore, we propose defining 10 features computed at the time-window level, specifically designed to remain discriminative even when attack traffic is mixed with a large volume of legitimate traffic.

These features will help improve the detection of certain types of DDoS attacks, despite the presence of high levels of normal traffic.

Feature 1: ratio_max_mean_pkts

This feature measures the asymmetry of the traffic within a window: does a single flow abnormally dominate the others?

How is it calculated?

Packet level — what is extracted:

- ip_src
- port_src

Flow level — what is calculated:

- pkts_count += 1 for each received packet
- At end of window: pkts_total_i = pkts_count_i

Window level — final calculation:

```
values = [pkts_total_1, pkts_total_2, ..., pkts_total_N]
ratio_max_mean_pkts = max(values) / mean(values)
             = max(pkts_total_i) / (Σ pkts_total_i / N)
```

Utility by Attack Percentage

Test scenario — 5-second window:

- Total flows = 1000
- Normal flows: average pkts_total = 12 packets
- Attack flows: pkts_total = 4500 packets

Case 1: Attack = 5% of flows (50 flows)

```
Normal: 950 × 12 = 11,400 packets
Attack: 50 × 4,500 = 225,000 packets
Total = 236,400 | mean = 236.4 | max = 4,500
ratio = 4,500 / 236.4 = 19.0  ← VERY HIGH (normal = 3-8)  ✓ DETECTED
```

Case 2: Attack = 50% of flows (500 flows)

```
Normal: 500 × 12 = 6,000 packets
Attack: 500 × 4,500 = 2,250,000 packets
Total = 2,256,000 | mean = 2,256 | max = 4,500
ratio = 4,500 / 2,256 = 2.0  ← BELOW THRESHOLD (< 3)  ✗ NOT DETECTED
```

Useful Attack Types

Concentrated DDoS:

- 1 to 20 source IPs send massively
- Few attack flows but each with very high pkts_total
- % attack flows << 5% → ratio_max_mean_pkts explodes

Amplification (DNS, NTP, Memcached):

- Attacker sends a small request
- Amplified server responds with a HUGE packet (factor ×50 to ×500)
- Amplification flows have very high total bytes AND high pkts_total

- ratio_max_mean_pkts captures the largest flow → always an amplification flow

Feature 2: pkts_total_p99

This feature measures the packet volume of the top 1% most active flows in the window.

Examples

Scenario 1: Normal traffic (1000 flows)

- 80% of flows: 2–10 packets
- 15% of flows: 11–25 packets
- 5% of flows: 26–45 packets
- N=1000 → rank_p99 = ceil(0.99×1000) = 990 → pkts_total_p99 = 45

Scenario 2: UDP Flood (20% of flows, 200 attack flows)

- Normal flows (800): pkts_total ∈ [2, 45]
- Attack flows (200): pkts_total = 4500
- rank_p99 = 990 → falls in attack zone (801–1000) → pkts_total_p99 = 4500 (+9900%)

Scenario 3: Attack = 1% of flows (10 attack flows)

- rank_p99 = 990 → still in the normal zone (1–990) → pkts_total_p99 = 45 (normal)
- Conclusion: Attack must represent more than 1% of flows to be captured by p99

How is it calculated?

Flow level:

```
pkts_total_i = pkts_count_i  (same as Feature 1)
```

Window level:

```
values = sorted([pkts_total_1, ..., pkts_total_N])
rank = ceil(0.99 × N)
pkts_total_p99 = values[rank]
```

Why p99?

- p99.5 looks at only 0.5% of flows → very sensitive to outliers → many false positives
- p99 looks at 1% of flows → more stable, less sensitive to outliers
- In practice, an effective DDoS attack often represents >5% of traffic
- For small attacks (<2%), other features (entropy, syn_ratio) capture them

Are Feature 1 and Feature 2 Complementary?

F2 has blind spots that F1 covers, and vice versa.

F2 Blind Spots (Covered by F1)

Blind Spot 1: Very small attack (<2% of flows)

- F2: p99 lands in normal zone → NOT detected

```
F1: max=4500, mean=56.88 → ratio=79.1  ← EXPLODES
```

Blind Spot 2: Moderate attack flows near normal range

- F2 may detect, but also generates false positives during normal traffic peaks

```
F1: ratio=2.78  ← WITHIN NORMAL RANGE → avoids false positive
```

Blind Spot 3: Attack volume not in p99 range

- F2: p99 lands in normal zone → NOT detected

```
F1: max=4500, mean=36.88 → ratio=122  ← EXPLODES
```

F1 Blind Spots (Covered by F2)

Blind Spot 1: Attack with many flows (20%+)

```
F1: ratio=4.95  ← WITHIN NORMAL RANGE → NOT detected
F2: p99 = 4,500  ← EXPLODES
```

Blind Spot 2: Distributed attack (many moderate flows)

```
F1: ratio=1.79  ← WITHIN NORMAL RANGE → NOT detected
F2: p99 = 100  ← HIGH (normal = 45)
```

Scenario	F2 Detects?	F1 Detects?	Most Useful
Attack < 2% of flows	NO	YES	F1
Attack 2–10% of flows	PARTIAL	YES	F1
Attack 10–30% of flows	YES	PARTIAL	F2
Attack > 30% of flows	YES	NO	F2
Distributed (moderate flows)	YES	NO	F2
Normal peak (e.g. Saturday evening)	□ FALSE POS.	CALM	F1
Attack: few very large flows	YES	YES	Both

Feature 3: bytes_total_p99

This is the byte volume of the top 1% of flows that send the most data.

F2 measures the number of packets; F3 measures the byte volume. These two metrics can be very different.

Example: Attack with Abnormally Large Packets

- Normal HTTPS flow: 1,000 packets × 1,500 bytes = 1,500,000 bytes
- Attack flow: 1,000 packets × 7,500 bytes = 7,500,000 bytes
- F2: 1,000 vs 1,000 → IDENTICAL → NOT DETECTED
- F3: 7,500,000 vs 1,500,000 → 5× more → DETECTED

How is it calculated?

Packet level:

- ip_src, port_src, p.size (packet size in bytes)

Flow level:

```
bytes_total_i = sum of p.size for all packets in flow i
```

Window level:

```
values = sorted([bytes_total_1, ..., bytes_total_N])
rank = ceil(0.99 × N)
bytes_total_p99 = values[rank]
```

Why Keep F3?

- Avoids false positives (if packet count peaks but volume is normal → F3 stays calm)
- Note: On the internet, packets > 1500 bytes don't exist because MTU = 1500 bytes. Beyond that: fragmentation or rejection.

Feature 4: proto_udp_share

This is the proportion of UDP packets in all traffic received by the server during the window.

Our server: HTTPS on port 443

Normal Traffic vs. UDP Flood

- Normal HTTPS traffic: TCP ≈ 99.9%, UDP ≈ 0.01–0.1%
- proto_udp_share normal = 0.001 to 0.01

- During UDP Flood: UDP = 80%, TCP = 20%
- proto_udp_share = 0.80 → EXPLODES

Technical Note

Even though the server listens on port 443 TCP (HTTPS) and NOT on port 443 UDP, UDP packets can still ARRIVE at the server. The firewall/router lets them through, and the server

must process them (even to ignore them), consuming CPU resources and potentially saturating bandwidth.

UDP Flood on port 443 is very common in DDoS attacks.

Why This Feature Resists Dilution

- Normal traffic may have some UDP (DNS, NTP), but the proportion is VERY low (<1%)
- Even if the attack represents only 10% of packets: normal 1% UDP + attack 10% UDP = 11% total → proto_udp_share = 0.11 → 11× more than normal → DETECTED

How is it calculated?

Window level — direct calculation via global counters:

```
For each packet: total_pkts += 1
                 if p.proto = UDP: udp_pkts += 1
proto_udp_share = udp_pkts / total_pkts
```

Feature 5: proto_icmp_share

This is the proportion of ICMP packets in all traffic received by the server during the 5-second window.

ICMP Flood (Ping flood) is one of the oldest DDoS attacks and is very effective at saturating bandwidth.

How is it calculated?

```
For each packet: total_pkts += 1
                 if p.proto = ICMP: icmp_pkts += 1
proto_icmp_share = icmp_pkts / total_pkts
```

Important Note on Server Configuration

- Some servers may be configured to ignore pings, but ICMP packets still ARRIVE and consume CPU resources and bandwidth — so proto_icmp_share remains a valid signal.

Feature 6: syn_ratio

This is the proportion of TCP packets that have the SYN flag set (connection initiation) in all traffic received by the server during the window.

Normal TCP Traffic (HTTPS)

A normal TCP connection:

- SYN (1 packet) → SYN-ACK → ACK → DATA (1–10 packets) → FIN → FIN-ACK → ACK
- Total: 8–30 packets, with only 1 SYN at the start
- syn_ratio normal = 1/8 to 1/30 = 0.03 to 0.12

SYN Flood Attack

- Attacker sends ONLY SYN packets, never completing the handshake
- Server keeps connections in 'half-open' state, exhausting resources
- syn_ratio = 1.0 (all packets are SYN)

Why This Feature Resists Dilution

- Even during normal traffic peaks, syn_ratio rarely exceeds 0.15–0.20
- To reach 0.30 would require an abnormally high ratio of SYN to data packets

How is it calculated?

```
For each TCP packet: tcp_pkts += 1
                     if p.flag_syn = 1: syn_pkts += 1
syn_ratio = syn_pkts / tcp_pkts  (0 if tcp_pkts = 0)
```

Feature 7: entropy_ip_src

This is a measure of the diversity of source IP addresses sending traffic to the server: the more different IPs there are, evenly distributed, the higher the entropy.

Situation	entropy_ip_src	Interpretation
Normal traffic	8–12 bits	Normal IP distribution
Concentrated attack (1–10 IPs)	0–3 bits	VERY LOW
Botnet / Distributed DDoS	12–16 bits	VERY HIGH (abnormally high)
Port scan	12–16 bits	VERY HIGH

- If entropy is too LOW → concentrated attack (1–10 IPs)
- If entropy is too HIGH → distributed attack or IP spoofing

IP Spoofing: attacker sends packets with random source IPs → entropy = log2(10,000) ≈ 13.3 bits → VERY HIGH → Attack DETECTED

How is it calculated?

```
For each packet: ip_src_counts[p.ip_src] += 1
total_pkts = sum of ip_src_counts.values()
For each IP k: p_k = ip_src_counts[k] / total_pkts
entropy_ip_src = -Σ p_k × log2(p_k)
```

Feature 8: pkt_arrivals_avg_p1

This is the average inter-arrival interval between packets for the 1% of flows that send packets the fastest.

Human / Normal Application Traffic

- User browsing: grouped packets (every 0.01–0.1s), then 2–5 second pauses
- Average inter-arrivals: 0.5s to 2.5s

Bot / Automated Attack Traffic

- Bot sending continuously: packet every 0.002s (2 milliseconds)
- Average inter-arrivals: 0.002s

What This Feature Captures

- Normal traffic — fastest legitimate flows: inter-arrivals ≈ 0.02–0.1s
- Automated attack — bots send at much higher frequency: inter-arrivals < 0.005s
- p1 drops drastically during attack
- Difference is a factor of 10 to 1000 — NEVER 0.0001–0.001s in legitimate traffic

Limitations

- Requires precise timestamps (nanoseconds) and high-performance network capture (DPDK, XDP, libpcap)
- A slow bot (1 packet/s): inter-arrival = 1s → within normal range → NOT detected by F9, but captured by F2 (volume) or F8 (short connections)

Feature 9: pkt_size_avg_p1

This is the average packet size for the 1% of flows that send the smallest packets.

Alone, pkt_size_avg_p1 cannot detect an attack, because normal traffic already contains small packets (ACK, initial SYN, keep-alive). The 1st percentile of flows is therefore often 40–60 bytes even in the absence of an attack.

True Utility: In Combination

- F6 (syn_ratio) → many SYN packets
- F8 (conn_pkts_avg_p1) → very short or incomplete connections
- F9 (pkt_arrivals_avg_p1) → packets sent very rapidly
- Together, F10 confirms that suspicious flows have small packets — characteristic of SYN/ACK Flood attacks

F10 is an essential feature to keep, but never used alone for detection. It acts as a confirmation signal within the full feature set.

How is it calculated?

```
Per flow: pkt_size_avg_i = mean(pkt_sizes_i) = Σ pkt_sizes_i / pkts_count_i
Window: rank = ceil(0.01 × N)
pkt_size_avg_p1 = sorted(values)[rank]
```

- Normal → p1 ∈ [150, 300] bytes | SYN flood → p1 ≈ 40–64 bytes

Why p99 and p1?

A percentile p is the value below which p% of observations fall. This allows detection of extremes (very low or very high values) without being influenced by the majority of normal flows.

- p1 → captures the lowest flows / very small packets / very short intervals
- p99 → captures the most extreme flows / very large volumes / massive fragments

p1 and p99 are extreme enough to capture significant anomalies, but not too sensitive to rare outliers.

Percentile Choice	Advantage	Disadvantage
p1 / p99	Captures extremes in a stable and robust manner	Slightly less sensitive to extremely rare flows (<1%)
p0.1 / p99.9	Captures very rare extreme values	Too sensitive to noise, unstable on medium-sized windows
p2 / p98	Closer to the mean	Signal dilution, less distinctive for extreme anomalies

Final Summary

Attack Type	Features That Detect DIRECTLY	Features That Confirm / Avoid False Positives
SYN Flood	F2 (pkts_total_p99), F6 (syn_ratio), F8 (conn_pkts_avg_p1), F9 (pkt_arrivals_avg_p1)	F10 (pkt_size_avg_p1), F1 (ratio if concentrated)
UDP Flood	F2, F4 (proto_udp_share)	F3 (bytes_total_p99), F1 (if concentrated)
ICMP Flood	F5 (proto_icmp_share)	F2, F3
Amplification	F3, F4	F2, F1 (if concentrated), F7 (low entropy_ip_src)
Concentrated HTTP Flood	F1, F2, F3	F7 (low entropy_ip_src)

Distributed HTTP Flood	F2, F3	F7 (high entropy_ip_src), F9 (if bots)
Botnet / Bots	F7, F9	F2, F3
Port Scan	F7, F8	F6, F9
Flash Crowd (legitimate)	(none)	F1, F7, F8 normal

Feature Roles Summary

Feature	Primary Role
F1 – ratio_max_mean_pkts	Packet concentration on a single flow
F2 – pkts_total_p99	Extreme packet count per flow
F3 – bytes_total_p99	Extreme total byte volume per flow
F4 – proto_udp_share	Share of UDP packets
F5 – proto_icmp_share	Share of ICMP packets
F6 – syn_ratio	SYN proportion → detects SYN flood
F7 – entropy_ip_src	Source diversity → detects botnets or flash crowds
F8– pkt_arrivals_avg_p1	Very short inter-arrival → detects bots / rapid flood
F9 – pkt_size_avg_p1	Very small packets → confirms SYN / ACK flood
F10-11 – heure_sin & heure_cos	show expected periodic behavior